

Járműorientáció becslés szintetikus képadathalmazon

Nagy Gergő Roland
AGKSD0

Tartalom

1. A feladat leírása	2
2. Elméleti háttér	2
2.1 Szintetikus adathalmazok szerepe	2
2.2 Járműorientáció-osztályozás	2
2.3 Képosztályozás konvolúciós neurális hálókkal	2
2.4 Transfer learning és ImageNeten előtanított modellek	3
2.5 Tanítási alapfogalmak: tensor, batch, epoch, loss, súlyok	3
2.6 Kiértékelési metrikák: accuracy, per-class accuracy, confusion matrix	3
3. A megvalósítás terve és kivitelezése	3
3.1 Projektfelépítés és fő komponensek	3
3.2 Adathalmaz-előállítás Blenderrel	4
3.3 Orientációs címkézés yaw szög alapján	4
3.4 Metaadatok és adathalmaz-struktúra	5
3.5 Image-level split	6
3.6 PyTorch Dataset és DataLoader	6
3.7 Modellek implementációja	6
3.8 Tanítási pipeline	7
3.9 Kiértékelési pipeline	7
3.10 Ábrakészítő eszköz	7
4. Tesztelés és eredmények	7
4.1 Tesztelési módszer	7
4.2 Tanítási és validációs görbék	7
4.3 Modellösszehasonlítás	10
4.4 Osztályonkénti pontosság	12
4.5 Konfúziós mátrixok értelmezése	13
4.6 Eredmények összefoglalása	16
5. Felhasználói leírás	17
5.1 Szükséges környezet	17
5.2 Adathalmaz előállítása	17
5.3 Split készítése	17
5.4 Modell tanítása	17
5.5 Modell kiértékelése	17
5.6 Modellek összehasonlítása és ábrák készítése	17
Irodalomjegyzék	17

1. A feladat leírása

A projekt célja egy olyan teljes képfeldolgozási folyamat bemutatása, amely szintetikus járműképek alapján négy orientációs osztály egyikét rendeli egy képhez. A megoldandó feladat tehát képosztályozási probléma: a rendszernek el kell döntenie, hogy a jármű közeledik, távolodik, balra halad vagy jobbra halad.

A feladat érdekessége, hogy a bemeneti adatok nem valós forgalmi felvételekből, hanem Blenderrel előállított képekből állnak. Ez kontrollált környezetet ad a teljes folyamat ellenőrzésére: a 3D modellekből kép készül, a yaw szögből címke származik, majd a modellek ugyanazon adathalmazon tanulnak és ugyanazon metrikák alapján hasonlíthatók össze.

2. Elméleti háttér

2.1 Szintetikus adathalmazok szerepe

Szintetikus adathalmaznak nevezzük azt az adatkorpuszt, amely nem közvetlenül mérésből vagy kézi gyűjtésből származik, hanem valamilyen szabályozott előállítási folyamat eredménye. Képfeldolgozási feladatoknál ilyen lehet a 3D jelenetből renderelt képanyag, ahol a kamera, a tárgy pozíciója, a megvilágítás és a címke is pontosan ismerhető.

A szintetikus képek előnye, hogy nagy mennyiségben, konzisztens címkézéssel készíthetők. A projektben ez különösen hasznos, mert a jármű orientációja a yaw szögből közvetlenül meghatározható. Ugyanakkor a szintetikus eredményeket óvatosan kell értelmezni, mert a valós képeken megjelenő háttér, zaj, takarás és megvilágítási változatosság nem feltétlenül jelenik meg ugyanilyen mértékben.

2.2 Járműorientáció-osztályozás

A járműorientáció-osztályozás azt vizsgálja, hogy egy képen látható jármű milyen irányban áll vagy mozog a kamera nézőpontjához képest. A projekt négy osztályt használ: *approaching*, *leaving*, *moving_left* és *moving_right*. Ezek a címkék egyszerű, de jól értelmezhető leírást adnak a jármű képbeli irányáról.

A feladat képalapú megközelítése azt jelenti, hogy a modell nem kap külön geometriai paramétereket vagy 3D információt. A bemenete egy renderelt kép, a kimenete pedig a négy osztályhoz tartozó valószínűségi becslés.

2.3 Képosztályozás konvolúciós neurális hálókkal

A konvolúciós neurális hálók olyan mélytanulási modellek, amelyek képi mintázatok felismerésére különösen alkalmasak. A konvolúciós rétegek lokális szűrőket alkalmaznak, ezért képesek élek, textúrák, alkatrészek és magasabb szintű objektumrészletek reprezentációját megtanulni.

Képosztályozásnál a modell a bemeneti képből jellemzőket nyer ki, majd ezek alapján osztályokat becsül. A kimeneti réteg annyi értéket ad, ahány osztály van. Ebben a projektben ez négy kimenetet jelent, amelyek a négy orientációs osztálynak felelnek meg.

2.4 Transfer learning és ImageNeten előtanított modellek

A transfer learning lényege, hogy egy nagy adathalmazon korábban megtanított modell tudását egy kisebb vagy speciálisabb feladaton használjuk fel. A projekt torchvision architektúrákat alkalmaz, amelyek ImageNeten előtanított súlyokkal indulhatnak.

Az előtanított modellek már megtanultak általános képi jellemzőket, például élek, formák és textúrák felismerését. A projektben a végső osztályozó réteg a négy járműorientációs osztályhoz igazodik, a tanítás pedig ehhez az új feladathoz hangolja a modellt.

2.5 Tanítási alapfogalmak: tensor, batch, epoch, loss, súlyok

A tensor többdimenziós numerikus adattároló. A képek a tanítás során tenzorokká alakulnak, jellemzően csatorna, magasság és szélesség dimenziókkal. A batch több kép együttese, amelyet a modell egy lépésben dolgoz fel.

Az epoch egy teljes áthaladást jelent a tanító adathalmazon. A loss, vagyis veszteség, azt méri, mennyire tér el a modell becslése a helyes címkétől. A súlyok a modell tanulható paraméterei; az optimalizáló ezeket módosítja úgy, hogy a veszteség csökkenjen.

2.6 Kiértékelési metrikák: accuracy, per-class accuracy, confusion matrix

Az accuracy az összes helyes becslés arányát mutatja. Egyszerűen értelmezhető mutató, de önmagában nem mindig elég, mert elrejtetheti, ha egyes osztályok nehezebbek a modell számára.

A per-class accuracy osztályonként mutatja a helyes találatok arányát. A confusion matrix pedig részletesen megmutatja, hogy a valódi osztályokból a modell melyik prediktált osztályba sorolt képeket. Ez különösen hasznos a tévesztési mintázatok értelmezéséhez.

3. A megvalósítás terve és kivitelezése

3.1 Projektfelépítés és fő komponensek

A projekt modulokra bontva kezeli az adathalmaz előállítását, a felosztást, a tanítást, a kiértékelést és az eredmények ábrázolását. A bemeneti források közül az assets/models/ mappa tartalmazza a 3D modelleket, a generated_dataset/ mappa pedig a Blenderrel előállított képeket és a hozzájuk tartozó metadata.csv fájlt.

A tanításhoz kapcsolódó részhalmazfájlok a split készítése után jönnek létre. A futtatás során létrejövő fájlok, például a metrikák, checkpointok, kiértékelési összefoglalók és ábrák a training_runs/ mappa alatt tárolhatók.

Szkript/modul	Feladat
src/generation/generate_dataset.py	Blenderben betölti a 3D modelleket, yaw szögek szerint renderel, majd képfájlokat és metadata.csv fájlt hoz létre.
src/training/split_dataset.py	A metadata.csv alapján train.csv, val.csv és test.csv részhalmazokat készít.
src/training/dataset.py	A képeket és a címkéket PyTorch Dataset formában olvassa be.

src/training/model.py	A támogatott torchvision modelleket négyosztályos kimenettel állítja össze.
src/training/train.py	Egy kiválasztott modellt tanít, és metrics.csv, konfigurációs fájlok, valamint checkpointok mentésével rögzíti az eredményeket.
src/training/evaluate.py	Betölti a best_model.pt állományt, majd evaluation.json és confusion_matrix.csv fájlt készít.
src/training/compare_models.py	A futtatások eredményeiből model_comparison.csv összesítést állít elő.
src/project_tools/generate_plots.py	Meglévő eredményfájlokból PNG ábrákat készít a training_runs/figures/ mappába.
src/project_tools/project_clean.py	A lokális munkakörnyezet tisztítását segíti, például ideiglenes vagy újra-előállítható fájlok eltávolításával.

Az adatok útja a projektben a következő: 1. a 3D modellek az assets/models/ mappában található; 2. a Blender képeket renderel a generated_dataset/ mappába; 3. a metadata.csv a relative_path, label, model_name és yaw_deg mezőket tárolja; 4. a split_dataset.py létrehozza a train.csv, val.csv és test.csv fájlokat; 5. a dataset.py beolvassa a képeket és címkéket; 6. a train.py betanít egy kiválasztott modellt, majd metrikákat és checkpointokat ír; 7. az evaluate.py betölti a best_model.pt fájlt, és evaluation.json, valamint confusion_matrix.csv fájlt készít; 8. a compare_models.py model_comparison.csv fájlt állít elő; 9. a generate_plots.py a meglévő eredményfájlokból PNG ábrákat készít.

3.2 Adathalmaz-előállítás Blenderrel

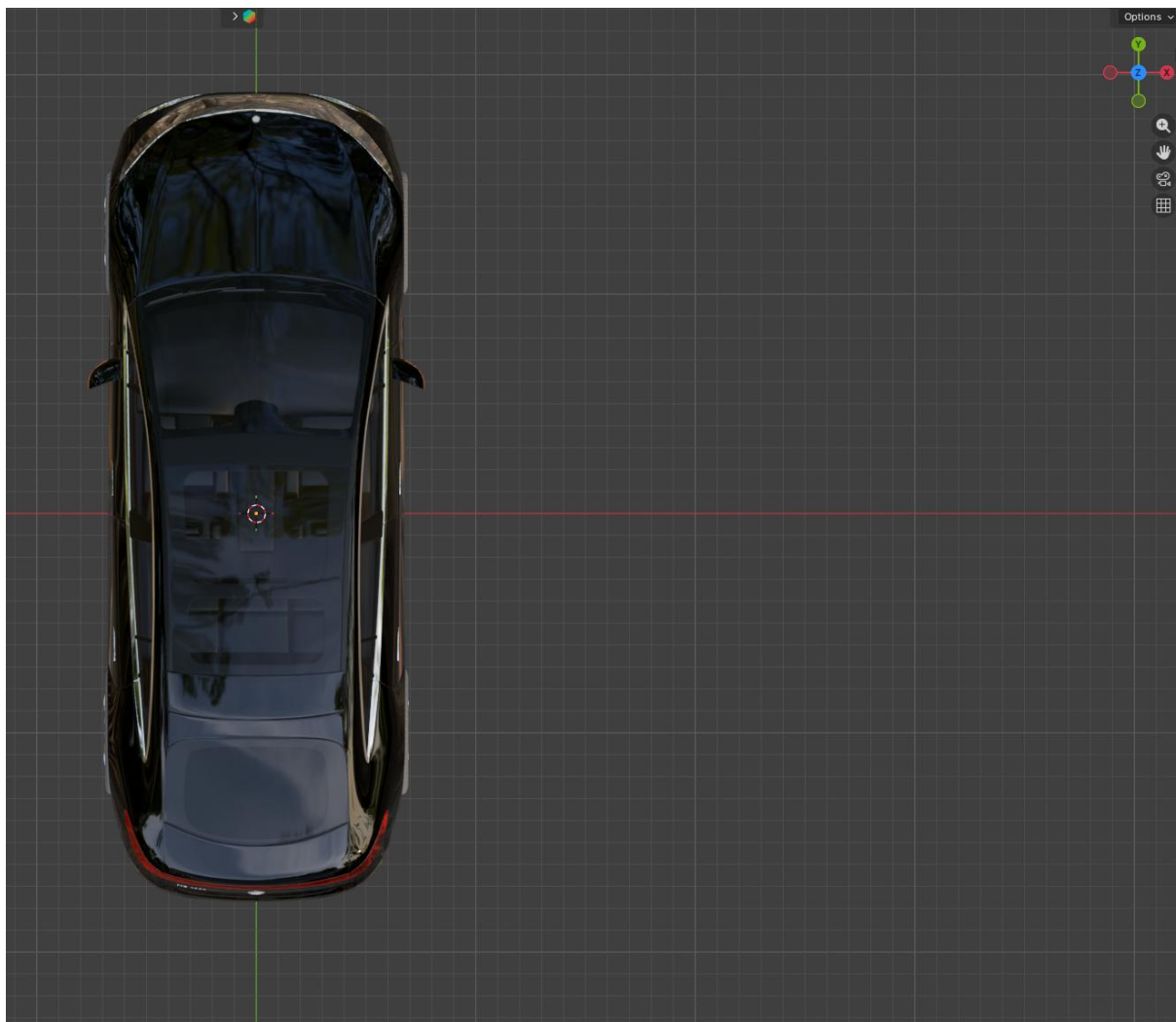
Az adathalmaz Blenderben készül. A folyamat egy 3D autómódellet betölt, normalizál, középre helyez, majd előre megadott yaw szögek szerint elforgat. Minden beállításnál kép készül, amely a megfelelő címkemappába kerül.

1. 3D modell betöltése az assets/models/ mappából;
2. a modell méretezése és jelenetbe helyezése;
3. yaw szög beállítása;
4. orientációs címke meghatározása;
5. kép renderelése a generated_dataset/ mappába;
6. metaadatsor rögzítése a metadata.csv fájlban.

A folyamat modelltől modellre halad. Egy adott jármű több nézőpontból is megjelenik, így ugyanazon objektumhoz több kép és több orientációs címke tartozhat.

3.3 Orientációs címkézés yaw szög alapján

A címkézés a Z tengely körüli yaw szög alapján történik. A projektben a jármű alaphelyzetben a +Y irányba néz, a kamera pedig a -Y oldalon található. Emiatt yaw 0 fok környékén a jármű távolodik a kamerától, yaw 180 fok környékén pedig közeledik.



1. ábra Blenderben egy autómodell és annak tájolása a különböző tengelyek mentén

Yaw tartomány	Címke	Jelentés
315-360 fok és 0-45 fok	leaving	a jármű távolodik a kamerától
45-135 fok	moving right	a jármű jobbra halad a képen
135-225 fok	approaching	a jármű közeledik a kamera felé
225-315 fok	moving left	a jármű balra halad a képen

3.4 Metaadatok és adathalmaz-struktúra

A generated_dataset/ mappa a renderelt képeket és a metadata.csv fájlt tartalmazza. A metadata.csv minden képhez rögzíti a relative_path, label, model_name és yaw_deg mezőket. A relative_path a kép helyét adja meg az adathalmazon belül, a label az orientációs osztály, a model_name a forrásul szolgáló 3D modell neve, a yaw_deg pedig a rendereléshez használt forgatási szög.

Ez a metaadat-struktúra teszi lehetővé, hogy a tanítási részhalmazok később újra-előállítható és ellenőrizhető módon készüljenek. A tanító kódnak nem kell a mappanevekből következtetnie minden információra, mert a legfontosabb mezők táblázatos formában is rendelkezésre állnak.

```
filename,relative_path,label,yaw_deg,model_name,model_path,image_width,image_height,camera_location,camera_target
audi_a7_yaw_000.png,leaving/audi_a7_yaw_000.png,leaving,0,audi_a7.glb,D:/JRM-O/assets/models/audi_a7.glb,512,512,"(0, -2.95143, 1.15)","(0, 0, 0.8)"
audi_a7_yaw_010.png,leaving/audi_a7_yaw_010.png,leaving,10,audi_a7.glb,D:/JRM-O/assets/models/audi_a7.glb,512,512,"(0, -3.21322, 1.15)","(0, 0, 0.8)"
audi_a7_yaw_020.png,leaving/audi_a7_yaw_020.png,leaving,20,audi_a7.glb,D:/JRM-O/assets/models/audi_a7.glb,512,512,"(0, -3.62013, 1.15)","(0, 0, 0.8)"
audi_a7_yaw_030.png,leaving/audi_a7_yaw_030.png,leaving,30,audi_a7.glb,D:/JRM-O/assets/models/audi_a7.glb,512,512,"(0, -3.91878, 1.15)","(0, 0, 0.8)"
audi_a7_yaw_040.png,leaving/audi_a7_yaw_040.png,leaving,40,audi_a7.glb,D:/JRM-O/assets/models/audi_a7.glb,512,512,"(0, -4.09868, 1.15)","(0, 0, 0.8)"
audi_a7_yaw_050.png,moving_right/audi_a7_yaw_050.png,moving_right,50,audi_a7.glb,D:/JRM-O/assets/models/audi_a7.glb,512,512,"(0, -4.56226, 1.15)","(0, 0, 0.8)"
audi_a7_yaw_060.png,moving_right/audi_a7_yaw_060.png,moving_right,60,audi_a7.glb,D:/JRM-O/assets/models/audi_a7.glb,512,512,"(0, -4.75364, 1.15)","(0, 0, 0.8)"
audi_a7_yaw_070.png,moving_right/audi_a7_yaw_070.png,moving_right,70,audi_a7.glb,D:/JRM-O/assets/models/audi_a7.glb,512,512,"(0, -4.89161, 1.15)","(0, 0, 0.8)"
audi_a7_yaw_080.png,moving_right/audi_a7_yaw_080.png,moving_right,80,audi_a7.glb,D:/JRM-O/assets/models/audi_a7.glb,512,512,"(0, -5.13377, 1.15)","(0, 0, 0.8)"
audi_a7_yaw_090.png,moving_right/audi_a7_yaw_090.png,moving_right,90,audi_a7.glb,D:/JRM-O/assets/models/audi_a7.glb,512,512,"(0, -5.36289, 1.15)","(0, 0, 0.8)"
audi_a7_yaw_100.png,moving_right/audi_a7_yaw_100.png,moving_right,100,audi_a7.glb,D:/JRM-O/assets/models/audi_a7.glb,512,512,"(0, -5.43002, 1.15)","(0, 0, 0.8)"
audi_a7_yaw_110.png,moving_right/audi_a7_yaw_110.png,moving_right,110,audi_a7.glb,D:/JRM-O/assets/models/audi_a7.glb,512,512,"(0, -5.3743, 1.15)","(0, 0, 0.8)"
audi_a7_yaw_120.png,moving_right/audi_a7_yaw_120.png,moving_right,120,audi_a7.glb,D:/JRM-O/assets/models/audi_a7.glb,512,512,"(0, -5.36022, 1.15)","(0, 0, 0.8)"
audi_a7_yaw_130.png,moving_right/audi_a7_yaw_130.png,moving_right,130,audi_a7.glb,D:/JRM-O/assets/models/audi_a7.glb,512,512,"(0, -5.18392, 1.15)","(0, 0, 0.8)"
audi_a7_yaw_140.png,approaching/audi_a7_yaw_140.png,approaching,140,audi_a7.glb,D:/JRM-O/assets/models/audi_a7.glb,512,512,"(0, -4.68777, 1.15)","(0, 0, 0.8)"
audi_a7_yaw_150.png,approaching/audi_a7_yaw_150.png,approaching,150,audi_a7.glb,D:/JRM-O/assets/models/audi_a7.glb,512,512,"(0, -4.36637, 1.15)","(0, 0, 0.8)"
audi_a7_yaw_160.png,approaching/audi_a7_yaw_160.png,approaching,160,audi_a7.glb,D:/JRM-O/assets/models/audi_a7.glb,512,512,"(0, -4.0745, 1.15)","(0, 0, 0.8)"
audi_a7_yaw_170.png,approaching/audi_a7_yaw_170.png,approaching,170,audi_a7.glb,D:/JRM-O/assets/models/audi_a7.glb,512,512,"(0, -4.13793, 1.15)","(0, 0, 0.8)"
audi_a7_yaw_180.png,approaching/audi_a7_yaw_180.png,approaching,180,audi_a7.glb,D:/JRM-O/assets/models/audi_a7.glb,512,512,"(0, -4.15329, 1.15)","(0, 0, 0.8)"
audi_a7_yaw_190.png,approaching/audi_a7_yaw_190.png,approaching,190,audi_a7.glb,D:/JRM-O/assets/models/audi_a7.glb,512,512,"(0, -4.12028, 1.15)","(0, 0, 0.8)"
audi_a7_yaw_200.png,approaching/audi_a7_yaw_200.png,approaching,200,audi_a7.glb,D:/JRM-O/assets/models/audi_a7.glb,512,512,"(0, -4.03955, 1.15)","(0, 0, 0.8)"
audi_a7_yaw_210.png,approaching/audi_a7_yaw_210.png,approaching,210,audi_a7.glb,D:/JRM-O/assets/models/audi_a7.glb,512,512,"(0, -4.36637, 1.15)","(0, 0, 0.8)"
audi_a7_yaw_220.png,approaching/audi_a7_yaw_220.png,approaching,220,audi_a7.glb,D:/JRM-O/assets/models/audi_a7.glb,512,512,"(0, -4.78061, 1.15)","(0, 0, 0.8)"
audi_a7_yaw_230.png,moving_left/audi_a7_yaw_230.png,moving_left,230,audi_a7.glb,D:/JRM-O/assets/models/audi_a7.glb,512,512,"(0, -5.26114, 1.15)","(0, 0, 0.8)"
audi_a7_yaw_240.png,moving_left/audi_a7_yaw_240.png,moving_left,240,audi_a7.glb,D:/JRM-O/assets/models/audi_a7.glb,512,512,"(0, -5.41483, 1.15)","(0, 0, 0.8)"
audi_a7_yaw_250.png,moving_left/audi_a7_yaw_250.png,moving_left,250,audi_a7.glb,D:/JRM-O/assets/models/audi_a7.glb,512,512,"(0, -5.40486, 1.15)","(0, 0, 0.8)"
audi_a7_yaw_260.png,moving_left/audi_a7_yaw_260.png,moving_left,260,audi_a7.glb,D:/JRM-O/assets/models/audi_a7.glb,512,512,"(0, -5.38803, 1.15)","(0, 0, 0.8)"
audi_a7_yaw_270.png,moving_left/audi_a7_yaw_270.png,moving_left,270,audi_a7.glb,D:/JRM-O/assets/models/audi_a7.glb,512,512,"(0, -5.29747, 1.15)","(0, 0, 0.8)"
audi_a7_yaw_280.png,moving_left/audi_a7_yaw_280.png,moving_left,280,audi_a7.glb,D:/JRM-O/assets/models/audi_a7.glb,512,512,"(0, -5.04671, 1.15)","(0, 0, 0.8)"
audi_a7_yaw_290.png,moving_left/audi_a7_yaw_290.png,moving_left,290,audi_a7.glb,D:/JRM-O/assets/models/audi_a7.glb,512,512,"(0, -4.76905, 1.15)","(0, 0, 0.8)"
```

2. ábra metadata.csv fájl egy részlete

3.5 Image-level split

A jelenlegi mérés image-level splitet használ. Ez azt jelenti, hogy az egyes képek kerülnek train, validation és test részhalmazokba. Így ugyanazon 3D modell különböző nézetei akár több részhalmazban is megjelenhetnek.

Ez a felosztás jól alkalmas a teljes folyamat ellenőrzésére, mert megmutatja, hogy az adathalmaz, a tanítás, a kiértékelés és az összehasonlítás együtt működik-e. A kapott pontosságokat azonban ennek megfelelően szintetikus image-level baseline eredményként kell értelmezni.

3.6 PyTorch Dataset és DataLoader

A dataset.py feladata, hogy a CSV fájlok alapján megtalálja a képeket, beolvassa a címkéket, és a bemenetet a modellek számára megfelelő formára alakítsa. A képek átméretezése, tenzorra konvertálása és normalizálása a tanítási folyamat része.

A DataLoader batch-eket készít a Dataset elemeiből. Ez hatékonyabb tanítást tesz lehetővé, és külön kezeli a train, validation és test részhalmazok bejárását.

3.7 Modellek implementációja

A model.py a támogatott architektúrák létrehozását fogja össze. A projektben ResNet18, MobileNetV3 Small, EfficientNet-B0 és DenseNet121 szerepel. Mindegyik modellnél a végső osztályozó réteg négy kimeneti osztályra illeszkedik.

A közös modellválasztó réteggel a tanítási és kiértékelési kód ugyanazt az interfészt használhatja, függetlenül attól, hogy éppen melyik architektúra fut.

3.8 Tanítási pipeline

A `train.py` egy kiválasztott modellt tanít a `train split` képein, miközben a `val split` alapján követi a tanulás alakulását. Az epochonkénti `loss` és `accuracy` értékek a `metrics.csv` fájlba kerülnek.

A legjobb validációs eredményhez tartozó súlyok `best_model.pt` néven menthetők. Ez az állomány lesz a kiértékelés bemenete, így a tesztelés nem feltétlenül az utolsó, hanem a validáció alapján legjobbnak bizonyult állapotot méri.

3.9 Kiértékelési pipeline

Az `evaluate.py` a `best_model.pt` checkpointot tölti be, majd a `test split` képein kiértékeli a modellt. A folyamat összesített tesztpontosságot, osztályonkénti pontosságot és konfúziós mátrixot számol.

Az eredmények `evaluation.json` és `confusion_matrix.csv` fájlokba kerülnek. Ezeket később a modellösszehasonlítás és az ábrakészítés is felhasználja.

3.10 Ábrakészítő eszköz

A `compare_models.py` több futtatás eredményeit foglalja össze `model_comparison.csv` fájlba. Ez a táblázat a modellek közötti összevetés alapja.

A `generate_plots.py` a meglévő `metrics.csv`, `evaluation.json`, `confusion_matrix.csv` és `model_comparison.csv` fájlokból készít PNG ábrákat. Ezek az ábrák a `training_runs/figures/` mappában találhatóak, és a tanulási görbék, pontosságok, osztályonkénti eredmények és konfúziós mátrixok áttekintését segítik.

4. Tesztelés és eredmények

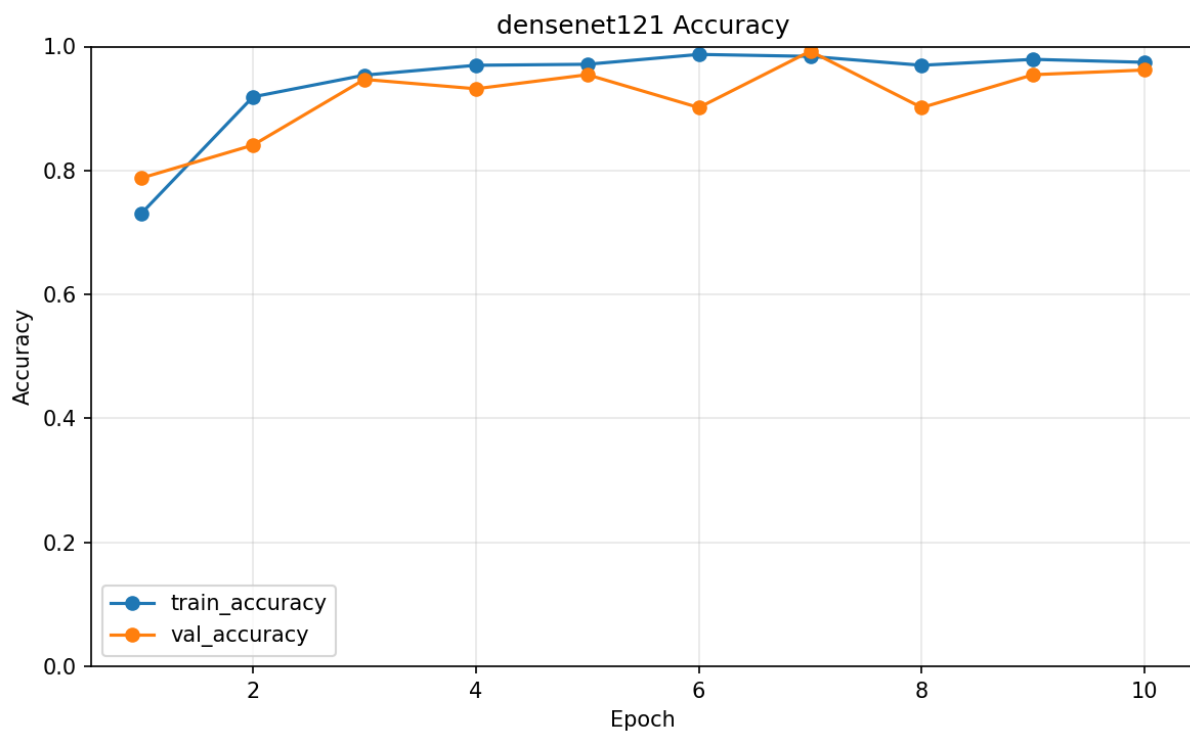
4.1 Tesztelési módszer

A tesztelés a `train`, `validation` és `test` felosztásra épül 70-15-15 százalékos felosztási arányban. A `train` részhalmaz a súlyok tanítására, a `validation` részhalmaz a tanulás követésére és a legjobb checkpoint kiválasztására, a `test` részhalmaz pedig a végső mérésre szolgál.

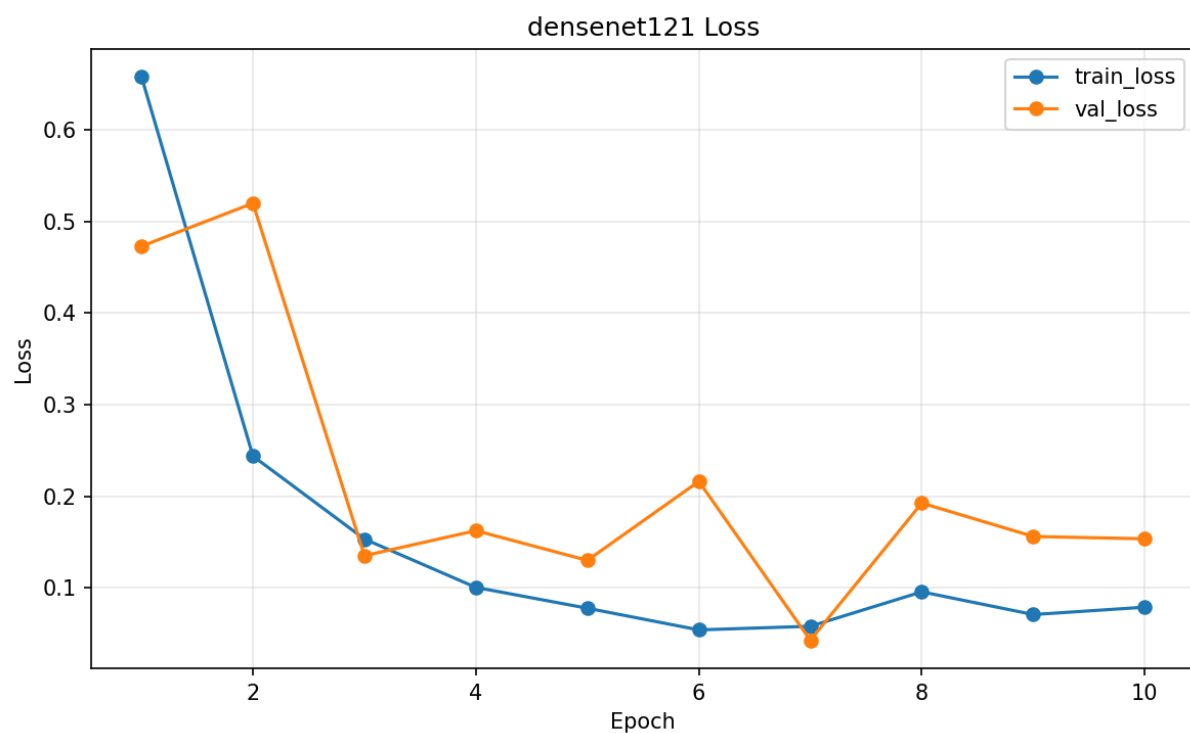
A modellek ugyanazon szintetikus image-level beállításon futottak, ezért az eredmények összehasonlíthatók egymással. Az értékek nem valós forgalmi teljesítményállítások, hanem a jelenlegi kontrollált adathalmazon mért baseline eredmények.

4.2 Tanítási és validációs görbék

A tanítási és validációs görbék azt mutatják, hogyan változott a modellek pontossága és vesztesége az epochok során. A `DenseNet121` görbéje gyors és stabil tanulást jelez, a veszteségábra pedig segít ellenőrizni, hogy a validációs teljesítmény nem csak véletlen ingadozás eredménye.



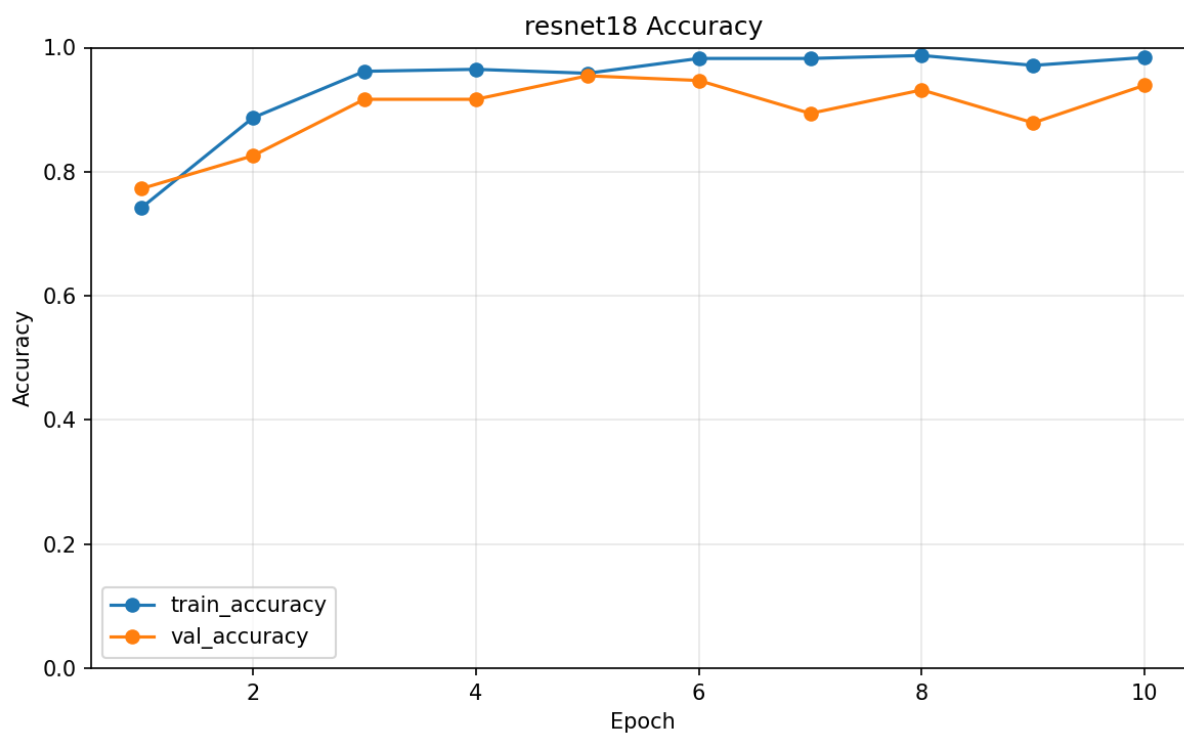
3. ábra DenseNet121 tanítási és validációs pontossága



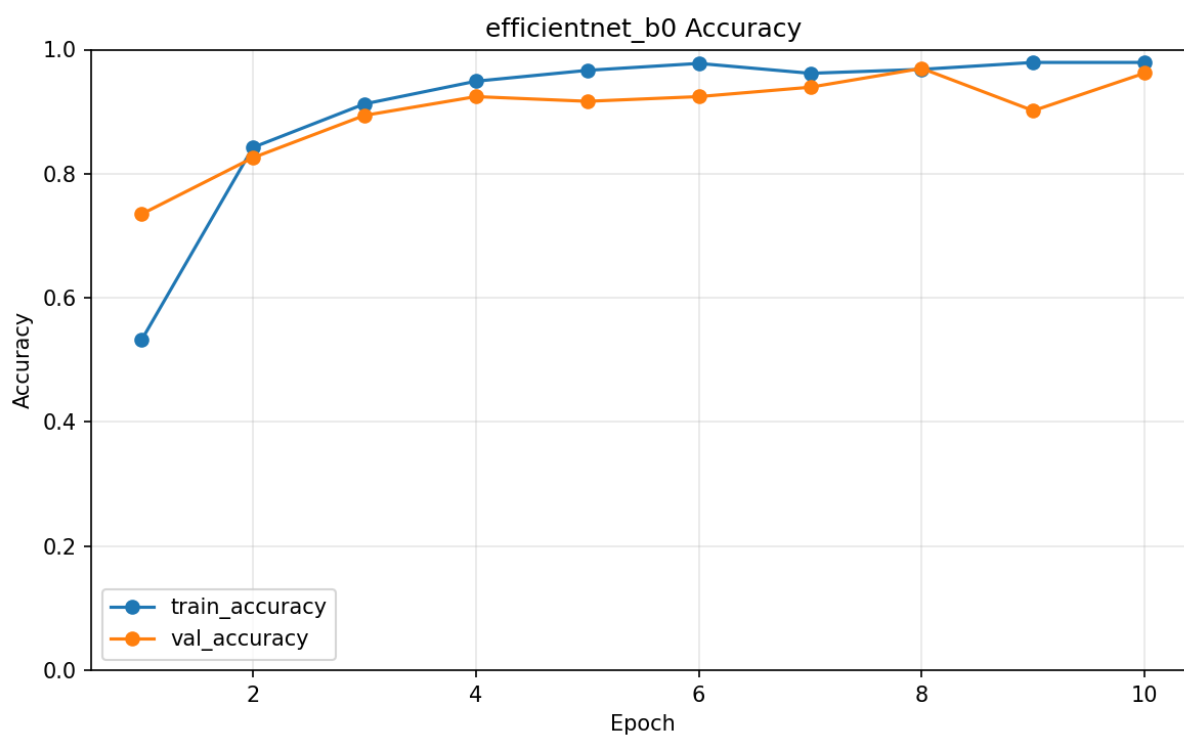
4. ábra DenseNet121 tanítási és validációs vesztesége

A többi modell pontossággörbéje alapján a ResNet18 stabilan magas validációs pontosságot ért el, az EfficientNet-B0 szintén erős eredményt adott, míg a MobileNetV3 Small kisebb

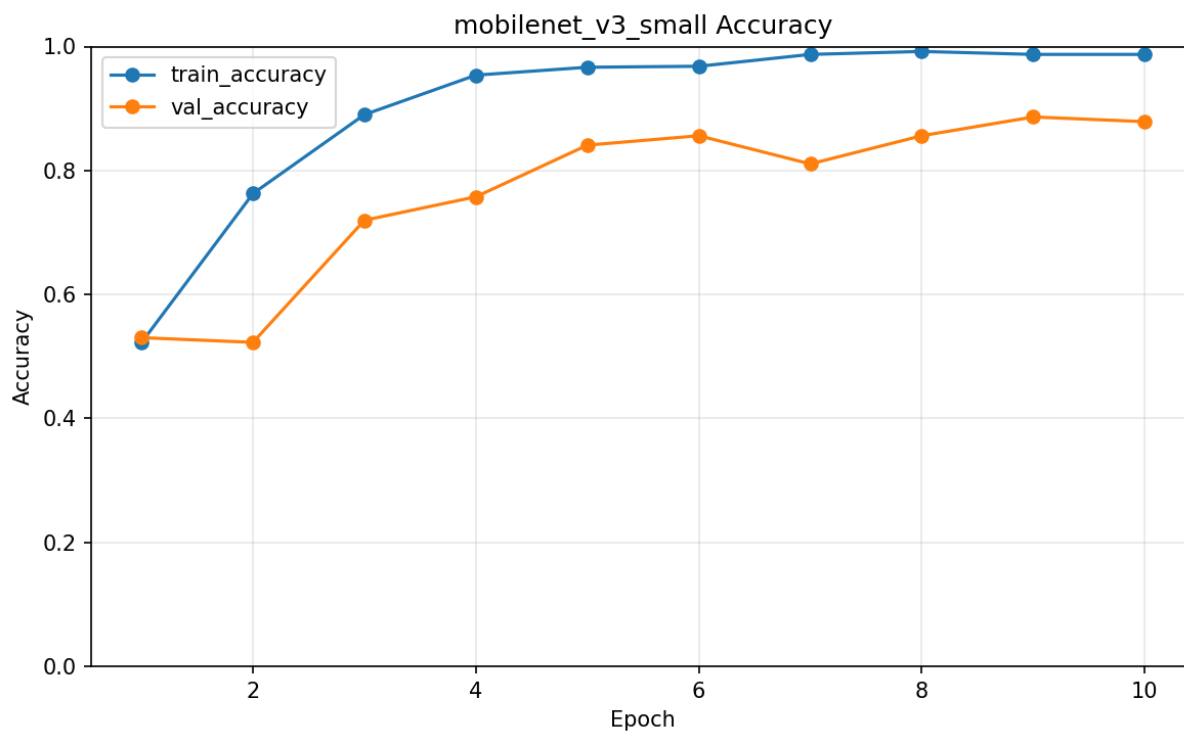
modellként alacsonyabb, de továbbra is használható teljesítményt mutatott.



5. ábra ResNet18 tanítási és validációs pontossága



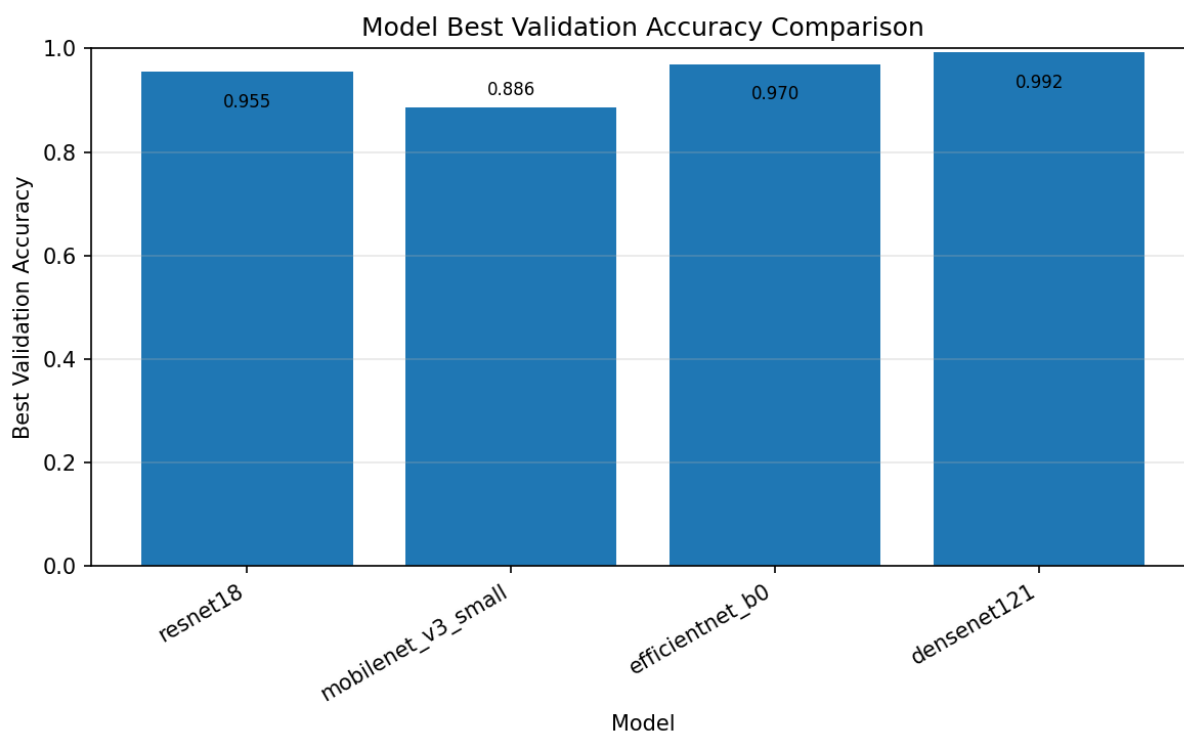
6. ábra EfficientNet-B0 tanítási és validációs pontossága



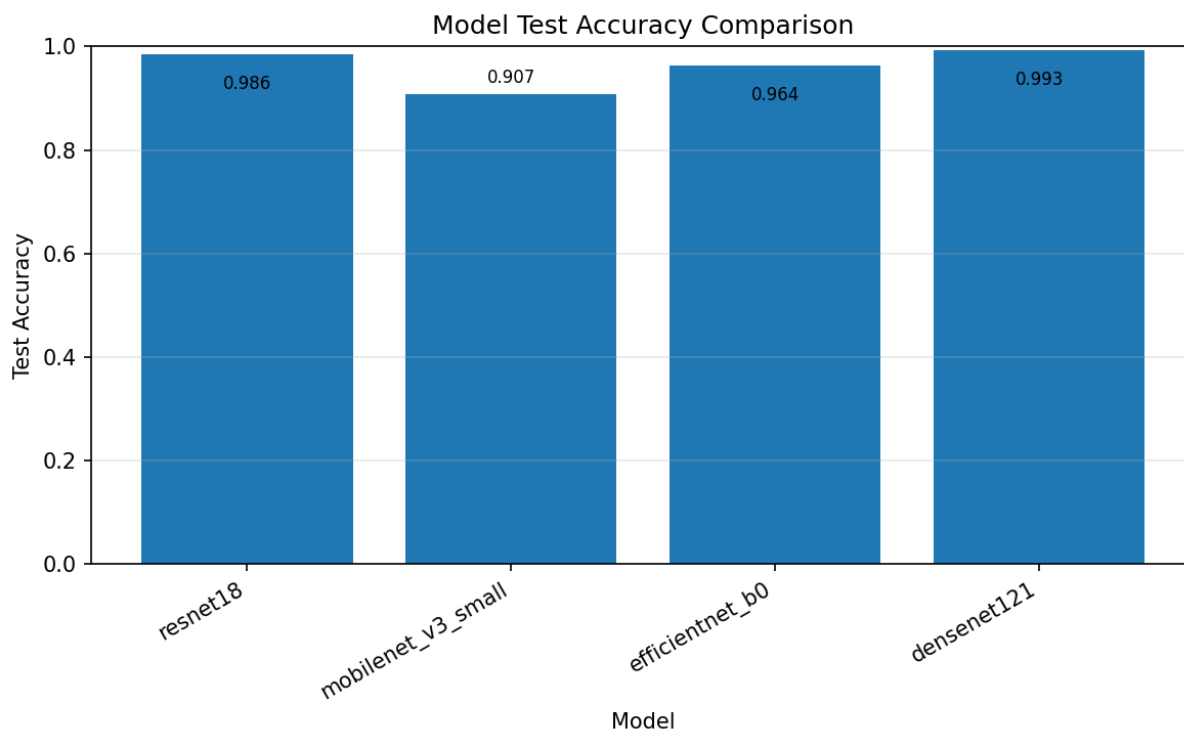
7. ábra MobileNetV3 Small tanítási és validációs pontossága

4.3 Modellösszehasonlítás

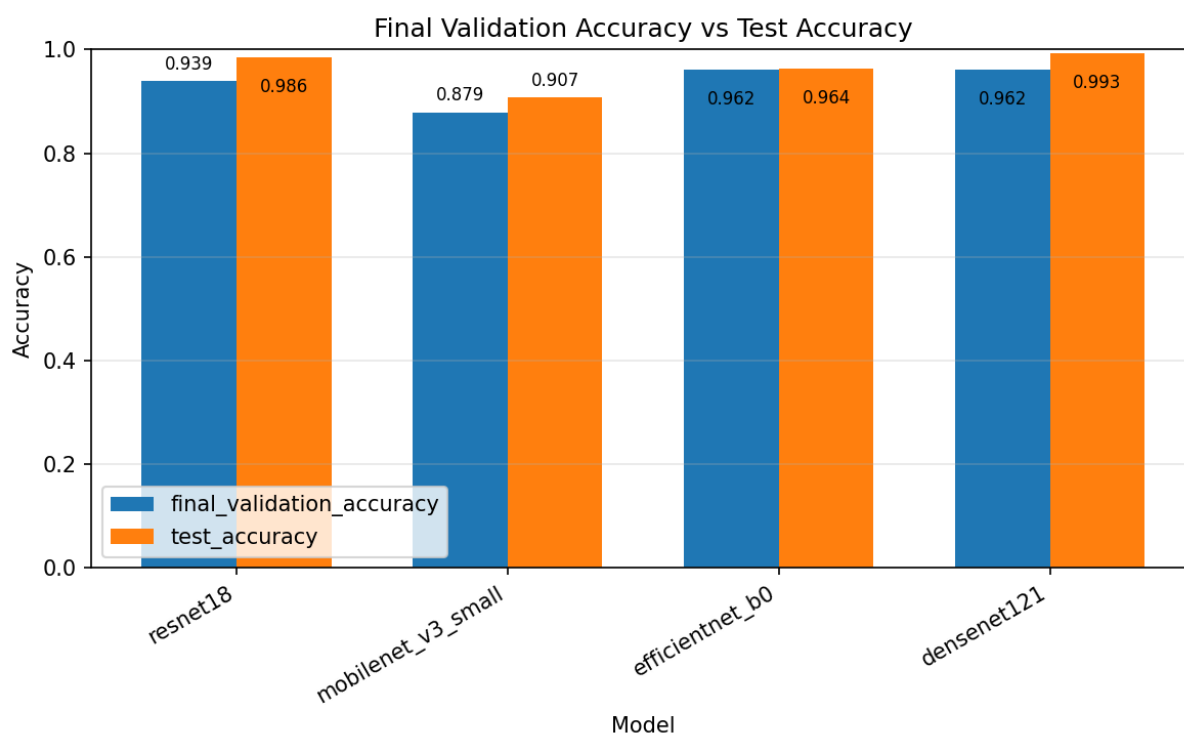
A modellek összehasonlítása a legjobb validációs pontosság és a tesztpontosság alapján történik. Az ábrák a fő értékeket foglalják össze.



8. ábra A négy modell validációs pontosságának összehasonlítása



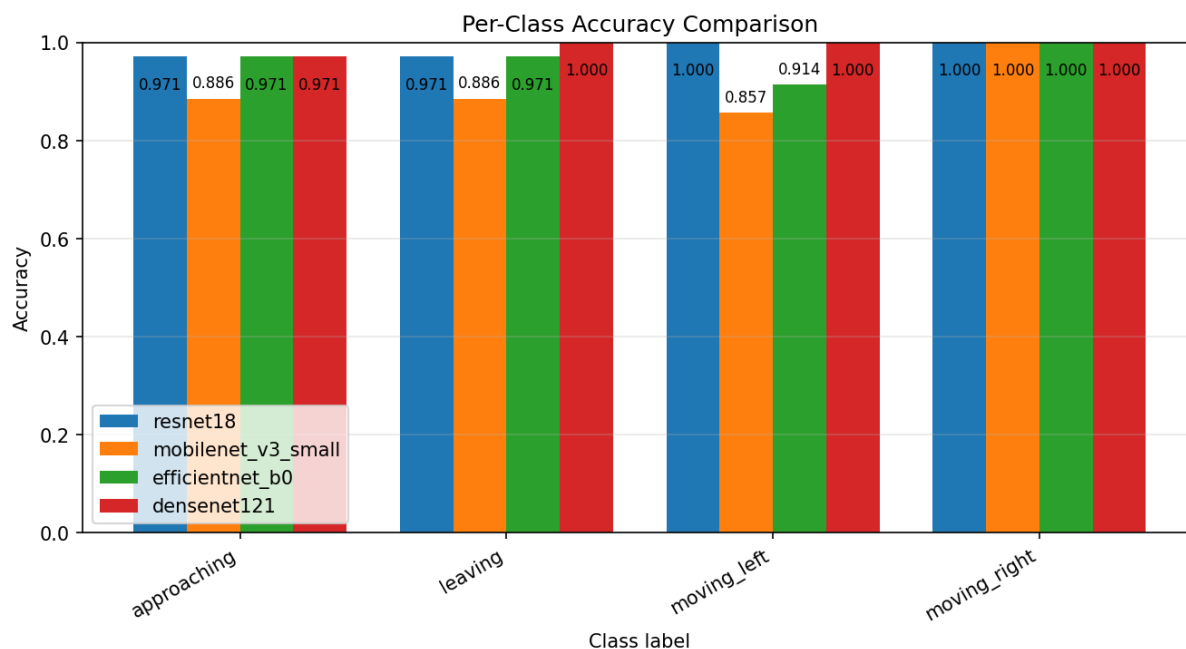
9. ábra A négy modell tesztpontosságának összehasonlítása



10. ábra A végső validációs és tesztpontossági értékek összevetése

4.4 Osztályonkénti pontosság

Az osztályonkénti pontosság részletesebben mutatja, mely irányok bizonyultak könnyűnek vagy nehéznek. A `moving_right` osztályt mindegyik modell hibátlanul osztályozta. A `moving_left` a MobileNetV3 Small és az EfficientNet-B0 számára volt a legnehezebb osztály.

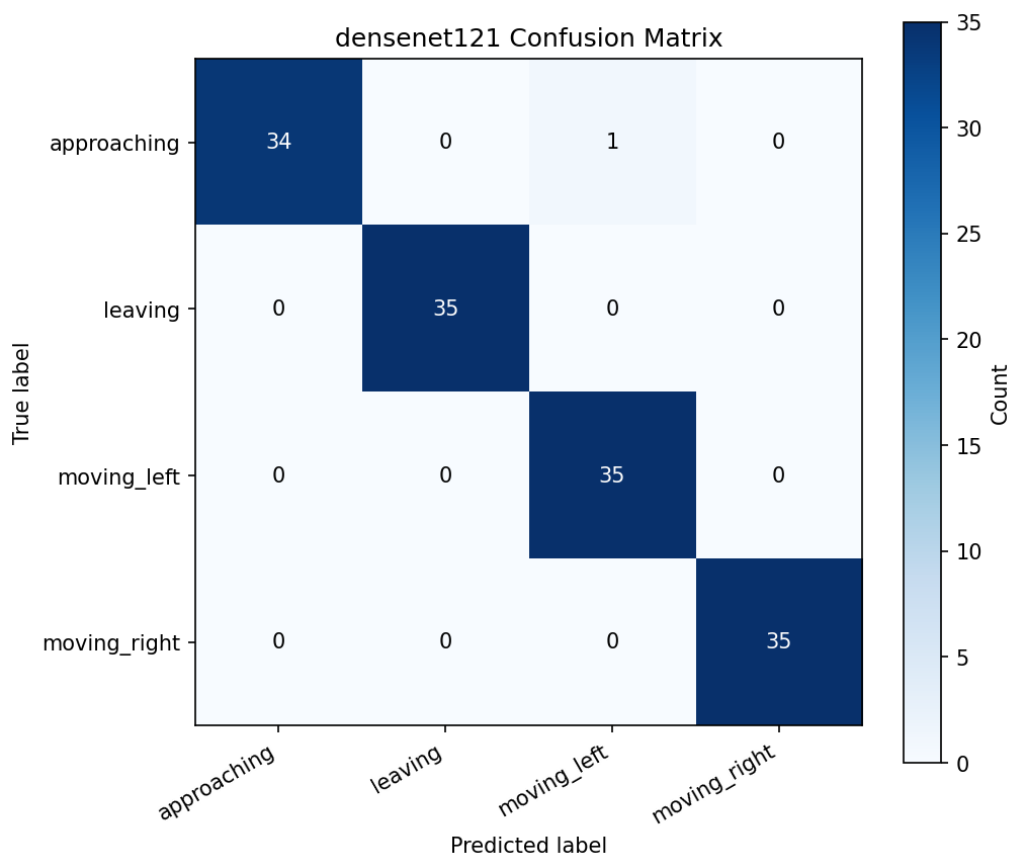


11. ábra Osztályonkénti tesztpontosság összehasonlítása

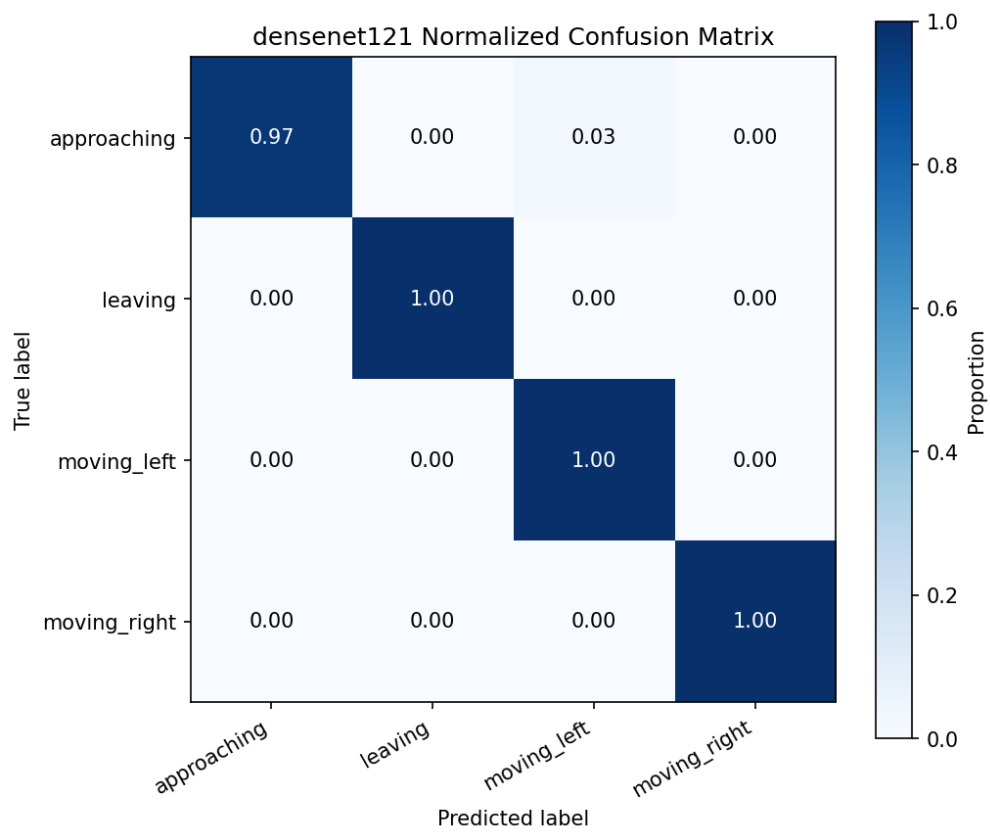
Ez az eredmény arra utal, hogy a négy osztály nem teljesen azonos nehézségű a modellek számára. A teljes pontosság mellett ezért érdemes mindig osztályonkénti bontást is vizsgálni.

4.5 Konfúziós mátrixok értelmezése

A konfúziós mátrixok megmutatják, hogy a hibák mely osztálypárok között jelennek meg. A DenseNet121 esetében nagyon kevés tévesztés látható, a normalizált változat pedig arányosan is szemlélteti az osztályonkénti teljesítményt.

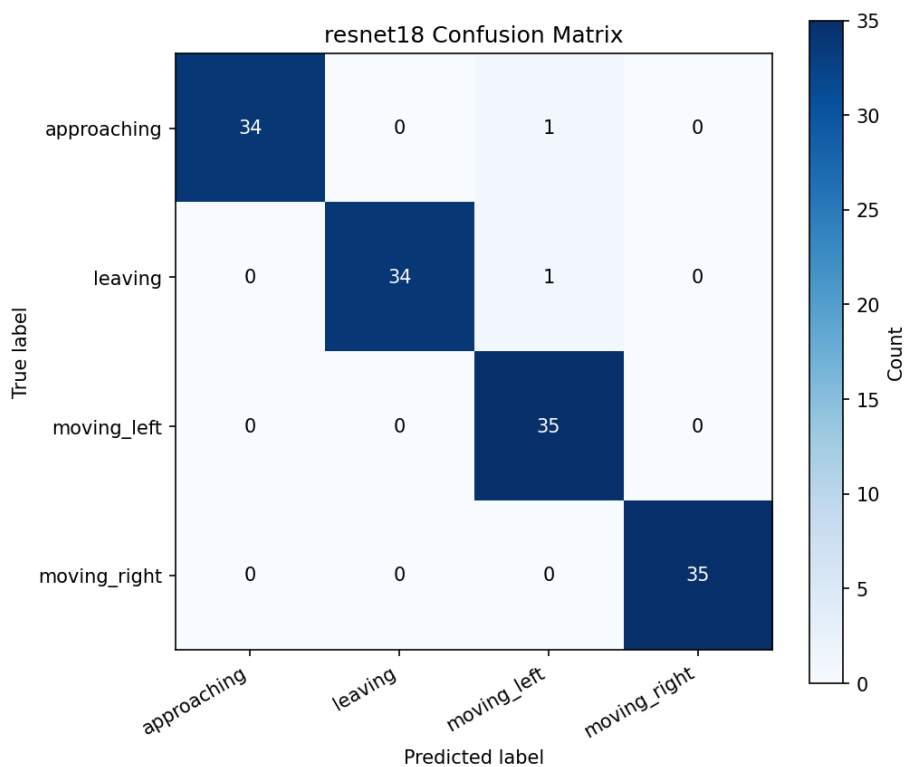


12. ábra A DenseNet121 nyers konfúziós mátrixa

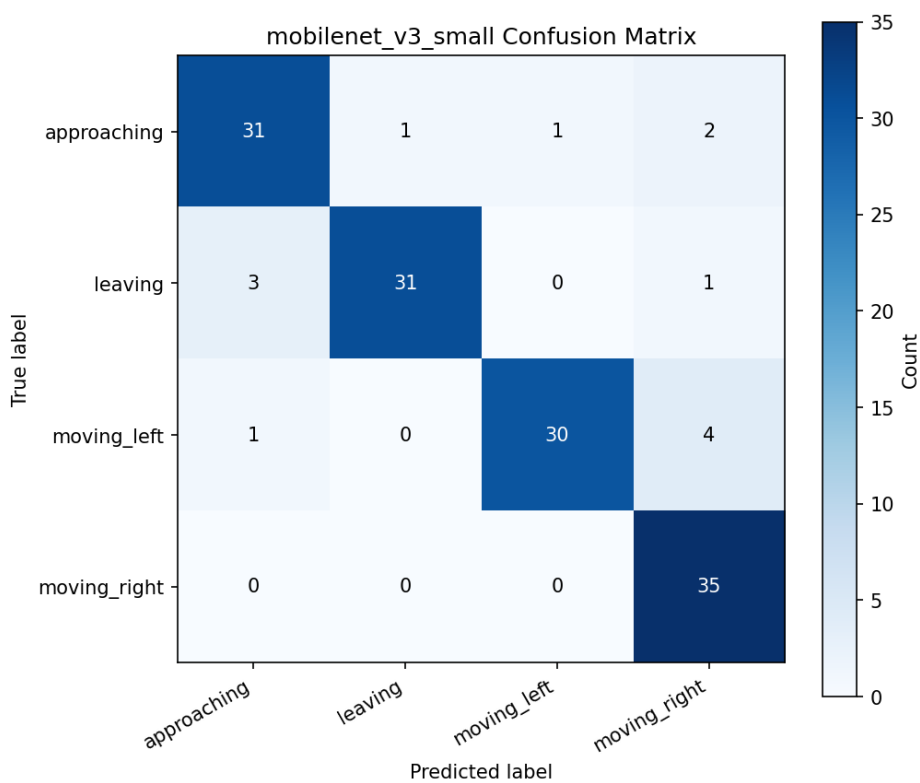


13. ábra A DenseNet121 normalizált konfúziós mátrixa

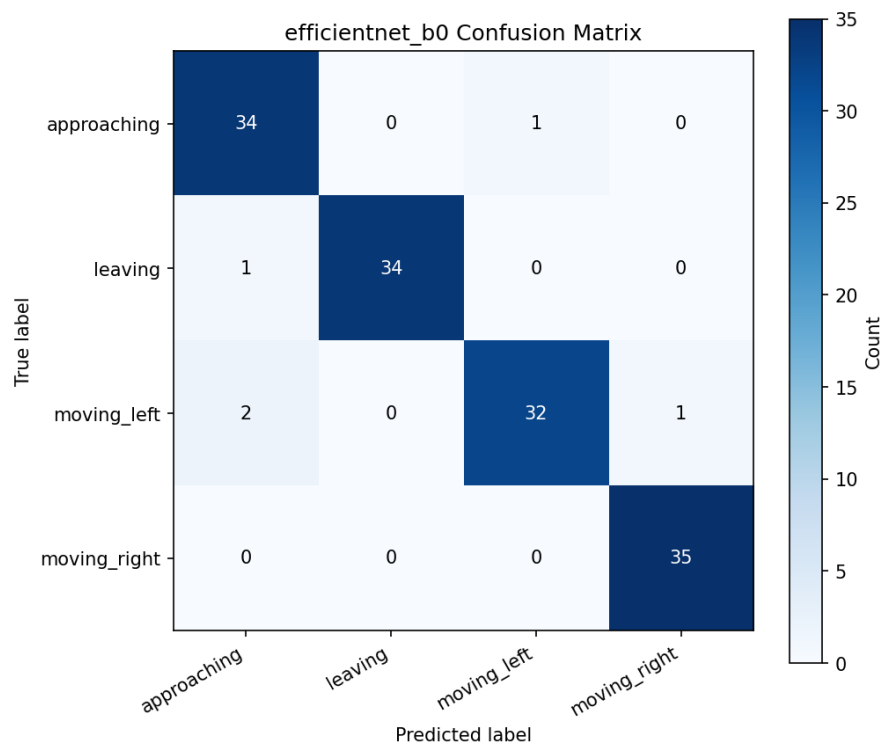
A ResNet18 mátrixa szintén erős teljesítményt jelez. A MobileNetV3 Small és az EfficientNet-B0 mátrixai jobban kirajzolják, hogy a moving_left osztály kezelése nehezebb volt ezeknek az architektúráknak.



14. ábra A ResNet18 konfúziós mátrixa



15. ábra MobileNetV3 Small konfúziós mátrixa



16. ábra EfficientNet-B0 konfúziós mátrixa

4.6 Eredmények összefoglalása

- A DenseNet121 teljesített a legjobban összességében.
- A ResNet18 szintén nagyon erős eredményt ért el.
- Az EfficientNet-B0 erős modell volt, de a moving_left osztálynál gyengébb eredményt adott.
- A MobileNetV3 Small volt a leggyengébb, de kisebb modellként továbbra is hasznos összehasonlítási alap.
- A moving_right osztályt mindegyik modell hibátlanul osztályozta.
- A moving_left a MobileNetV3 Small és az EfficientNet-B0 számára volt a legnehezebb.
- Az eredmények szintetikus image-level baseline mérések, nem valós forgalmi teljesítményállítások.

5. Felhasználói leírás

5.1 Szükséges környezet

A futtatáshoz Python környezet, PyTorch, torchvision és a projekt többi Python függősége szükséges. Az adathalmaz előállításához Blender is kell, mert a képek renderelése Blender szkripten keresztül történik.

```
python -m venv .venv .\venv\Scripts\activate pip install -r requirements.txt
```

A Blendernek telepítve kell lennie, és célszerű úgy beállítani, hogy a blender parancs elérhető legyen a terminálból.

5.2 Adathalmaz előállítása

```
blender --background --python src/generation/generate_dataset.py -- --config configs/generation_config.json --models-dir assets/models --output-dir generated_dataset
```

5.3 Split készítése

```
python src/training/split_dataset.py --dataset-dir generated_dataset --output-dir splits --seed 42
```

5.4 Modell tanítása

```
python src/training/train.py --model-name resnet18 --dataset-dir generated_dataset --splits-dir splits --output-dir training_runs/resnet18_run_001 --epochs 10 --batch-size 16 --seed 42
```

5.5 Modell kiértékelése

```
python src/training/evaluate.py --model-name resnet18 --dataset-dir generated_dataset --splits-dir splits --checkpoint training_runs/resnet18_run_001/best_model.pt --batch-size 16
```

5.6 Modellek összehasonlítása és ábrák készítése

```
python src/training/compare_models.py --runs training_runs/resnet18_run_001 training_runs/mobilenet_v3_small_run_001 training_runs/efficientnet_b0_run_001 training_runs/densenet121_run_001 --output training_runs/model_comparison.csv
```

```
python src/project_tools/generate_plots.py --runs training_runs/resnet18_run_001 training_runs/mobilenet_v3_small_run_001 training_runs/efficientnet_b0_run_001 training_runs/densenet121_run_001 --comparison training_runs/model_comparison.csv --output-dir training_runs/figures
```

Irodalomjegyzék

- [1] “Blender 5.1 Manual.” Accessed: May 18, 2026. [Online]. Available: <https://docs.blender.org/manual/en/latest/>
- [2] “Matplotlib documentation — Matplotlib 3.10.9 documentation.” Accessed: May 18, 2026. [Online]. Available: <https://matplotlib.org/stable/>
- [3] “Models and pre-trained weights — Torchvision 0.27 documentation.” Accessed: May 18, 2026. [Online]. Available: <https://docs.pytorch.org/vision/stable/models.html>
- [4] “Python 3.14 documentation,” Python documentation. Accessed: May 18, 2026. [Online]. Available: <https://docs.python.org/3/>
- [5] “PyTorch documentation — PyTorch 2.12 documentation.” Accessed: May 18, 2026. [Online]. Available: <https://docs.pytorch.org/docs/stable/index.html>

